

UNIVERSITY OF BUEA

REPUBLIC OF CAMEROON

FACULTY OF ENGINEERING
AND TECHNOLOGY

PEACE-WORK-FATHERLAND

DEPARTMENT OF COMPUTER
ENGINEERING



COURSE TITLE:

**INTERNET PROGRAMMING (J2EE) AND MOBILE
PROGRAMMING**

BridgeLearn

COURSE CODE: CEF 440

NAMES	MATRICULE
BERINYUY GILLES KILALON	FE23A024
ITIE LLOYD BONYEKI	FE23A069
NGUEFACK ELVIRA DIKIELLE	FE23A112
NKWAIN REMI KUMA	FE23A123
NKENGLEFAC DORENOEL	FE23A120

**COURSE INSTRUCTOR
DR. NKEMENI VALERY**

TASK 5: UI DESIGN AND IMPLEMENTATION

I. INTRODUCTION:

This task focuses on the design and implementation of the user interface for the BridgeLearn mobile app. Building on system models developed in Task 4, this task brings the app to life through visual design and front-end development. It covers the creation of the app's identity, layout of key screens, and implementation of interactive components to ensure a smooth and Quality of Experience and Continuous Learning.

Before any screen was sketched, and before any component was written, the team established a unified identity foundation. This was not optional. Without it, five developers working in parallel produce five visually inconsistent screens. With it, every screen feels like it belongs to the same product.

II. APP IDENTITY

Brand Foundation

BridgeLearn's name communicates its purpose directly. It is a bridge — between students and knowledge, between poor connectivity and educational content. The word "Learn" grounds it in its function. The name was chosen primarily for its resonance with the platform's adaptive mission.

Colour System

The colour palette was derived from the SRS design system documented in Task 2 and extended into a six-colour named system that covers every visual state the platform requires.

Primary : Navy (#1A3C6E). Used for the top navigation bar, primary action buttons, active tab indicators, and all heading-level elements. Navy communicates authority and trust without the coldness of pure black.

Secondary : Teal (#0D9488). Used for secondary actions, enrolment badges, progress indicators, and success-adjacent UI elements. Teal pairs with navy to create a palette that reads as modern and accessible on both high-end and low-end Android displays.

Accent : Amber (#F0A500). Used exclusively for live session indicators, offline warning banners, and time-sensitive UI elements. Amber creates urgency without alarm. A pulsing amber badge communicates "this is live" in a way that is immediately understood cross-culturally.

Background : (#F8FAFC). A near-white off-white that reduces eye strain on extended reading sessions. Chosen specifically over pure white because it is more comfortable for text-heavy screens like the transcript view in the Content Viewer.

Surface : White (FFFFFF). Applied to cards, modals, and elevated content containers. The distinction between Background and Surface creates natural visual hierarchy without requiring borders or shadows on every element.

Error : (#DC2626). Standard accessible red, used only for validation errors and failure states.

Success : (#059669). Used for completion indicators, passing quiz scores, and confirmation states.

Typography Scale

The platform uses Inter as its primary typeface across both the React web dashboard and the Expo React Native mobile application. Inter was chosen for three reasons.

- First, it renders clearly at small sizes on low-end Android screens, which is critical for performance on budget devices.
- Second, it is free and open-source, removing any licensing consideration.
- Third, it has excellent support for both Latin characters used in English and French — the two languages the platform must serve as specified in our NF Requirements.

The scale is fixed: H1 at 28pt for screen-level titles, H2 at 22pt for section headers, H3 at 18pt for subsection titles and card headers, Body at 14pt for all reading content, and Caption at 12pt for metadata, labels, and secondary information. These sizes were validated against the 360×640px minimum viewport requirement. At 360px wide, a 14pt body text set in Inter allows approximately 45 characters per line, which is within the optimal reading range of 45–75 characters.

Iconography and Component Tone

Icons are functional, not decorative. The platform uses a minimal icon set for navigation and content type indicators: a play icon for video, a headphone icon for audio, a document icon for text content, and a signal icon for live sessions. These icons are supplemented by text labels on all navigation elements to ensure accessibility and bilingual clarity.

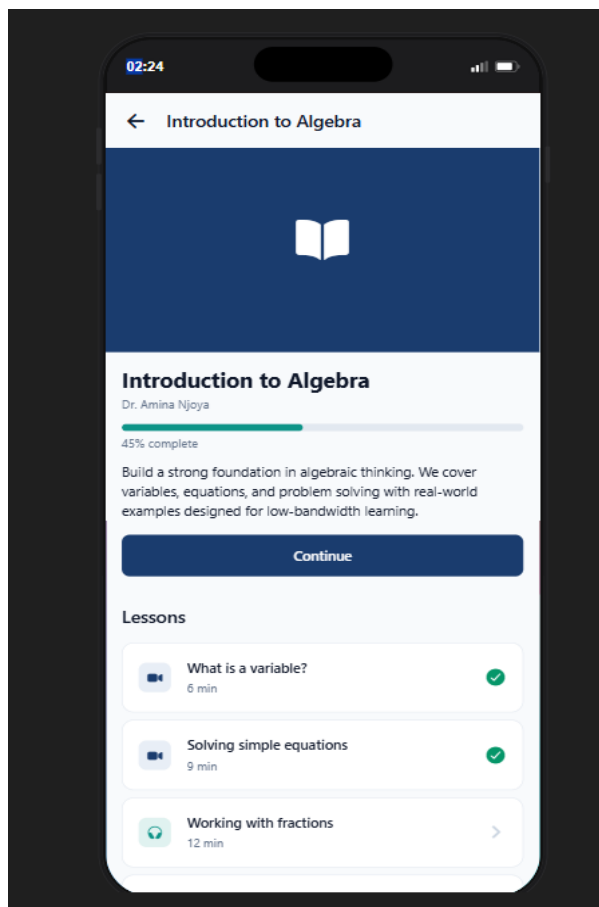


Figure 2 Student Course content

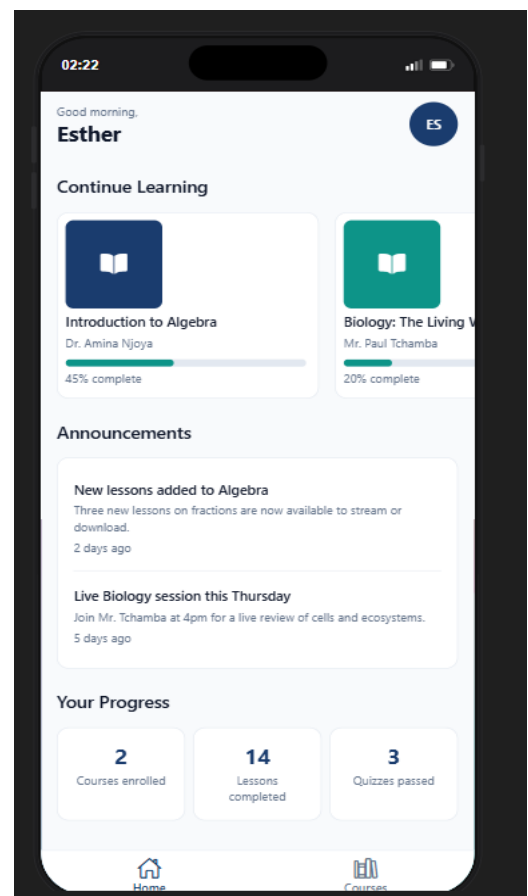


Figure 1 Student Dashboard

III. VISUAL DESIGN

The process for each screen followed five steps: rough paper sketch to agree on layout before opening Figma; high-fidelity Figma mockup using the identity system; requirements review to trace the screen back to a functional requirement in the SRS; implementation in React using hardcoded data first, then connected to the API; and testing against the platform's real constraints: low bandwidth simulation, 360px viewport, French language strings, and offline state.

Screen Inventory and Prioritisation

The full screen inventory was derived from the Task 4 use case diagram. Every use case maps to at least one screen. The team classified each screen as MVP, Phase 2, or Phase 3.

MVP screens are those without which the platform cannot satisfy its core functional requirements. These are Login, Register, Student Home Dashboard, Course Browser, Course Detail, Content Viewer, Live Lecture Viewer, Quiz Screen, Quiz Results, Instructor Dashboard, Course Creator, Content Uploader, and Live Session Launcher.

Phase 2 screens add completeness but do not block core functionality: Progress Dashboard, Offline Downloads Manager, Settings, Data Usage Tracker, Student Analytics View, User Management, System Health Monitor, and QoE Analytics Dashboard.

This prioritisation directly shaped the build sequence.

Key Screen Designs

Authentication Screens

The Login and Register screens establish the shared component library. Every interactive element built here — the primary button, text input with inline error states, the secondary button — is reused across all subsequent screens. The login screen is intentionally minimal: logo, two inputs, one primary action. The register screen introduces the role selector, implemented as toggle chips rather than a dropdown, because toggle chips are faster to interact with on a touchscreen and communicate the two available roles more clearly to a new user.

Both screens include an offline banner — a full-width amber strip — that appears automatically when the device has no network connection. On the login screen, the form remains usable when offline, with submissions queued for when connectivity returns.

Content Viewer

The Content Viewer is BridgeLearn's most important screen. It is the direct expression of FR-09 (adaptive bitrate streaming), FR-10 (audio-only mode), FR-11 (text transcript mode), and FR-16 (offline content access). The screen exists in four distinct modes that are entered automatically based on measured network conditions, with a manual override always available.

In Video mode, the student sees a 16:9 video player with a quality badge showing the current resolution (480p or 240p) and a manual quality selector allowing them to force a lower mode. In Audio mode, the video player is replaced by a large centred playback circle and the

transcript text scrolls below it — the student receives the lecture content at a fraction of the bandwidth. In Text mode, the screen becomes a full-screen reading experience with the transcript and an estimated read time. In Offline mode, the system serves whatever was cached locally, with a banner indicating when the content was downloaded.

Mode transitions are non-disruptive. When the system detects a bandwidth drop and switches from video to audio, a brief non-blocking notification appears: "Switched to audio — saving data." The student is not interrupted; the lecture continues.

Quiz Screen and Offline Handling

The Quiz screen implements FR-16 at the quiz layer. A student who begins a quiz and loses connectivity mid-attempt can continue. Their answers are stored locally using the `OfflineSyncManager`. When they submit, the platform responds with "Answers saved — will submit when you're back online." When connectivity is restored, the batch submission endpoint (`POST /api/quizzes/batch-submit`) is called automatically. The server timestamps the submission with the original local time, preserving deadline compliance for students whose connection dropped near a deadline.

Instructor Dashboard and Content Uploader

The Instructor Dashboard uses a fixed left sidebar navigation pattern appropriate for desktop use. Instructors are assumed to be operating from a computer, not a phone, which allows a more information-dense layout than the student mobile-first screens. The dashboard surfaces key metrics — courses created, total enrolled students, live sessions held, active quizzes — and provides immediate access to course management.

The Content Uploader is among the more technically complex screens to design because it has the most states: idle, file selected, uploading with progress, processing (video transcoding is asynchronous), ready, and error. The design decision to separate "uploading" from "processing" was important. An instructor who uploads a 200MB video file needs to understand that their upload completing is not the same as the content being live. The processing status indicator — Pending → Processing → Ready — makes this pipeline visible and prevents support requests from instructors wondering why their just-uploaded video is not appearing to students.

IV. FRONTEND IMPLEMENTATION

Technology Decisions

The frontend implementation spans two applications: a React web dashboard built with Vite, and a React Native mobile application built with Expo. Both applications share a common package `@bridgelearn/shared` which contains the API client, authentication logic, type definitions, and utility functions. This monorepo structure, managed with npm workspaces, means that a change to the API client propagates to both applications simultaneously and is never implemented twice.

React was chosen for the web dashboard for its component model, its ecosystem maturity, and its compatibility with the team's existing knowledge base. Expo was chosen for the mobile application because it abstracts the native build toolchain, which is a significant advantage for

a student team that does not have macOS machines available for iOS development and needs to target Android primarily.

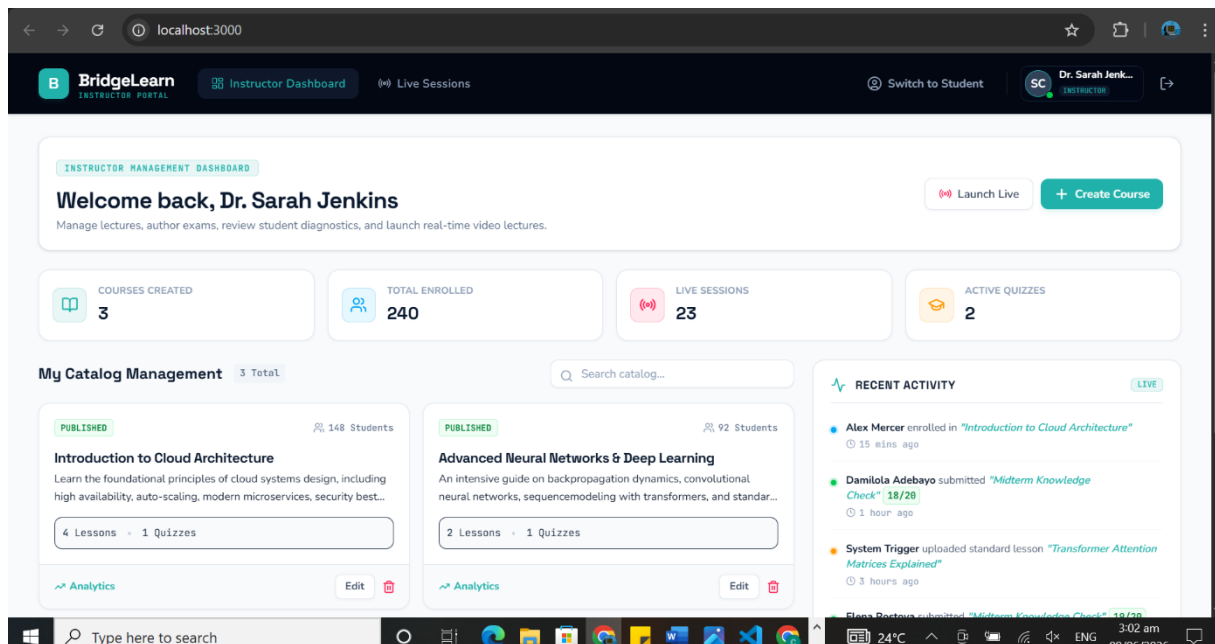


Figure 3 instructor Dashboard

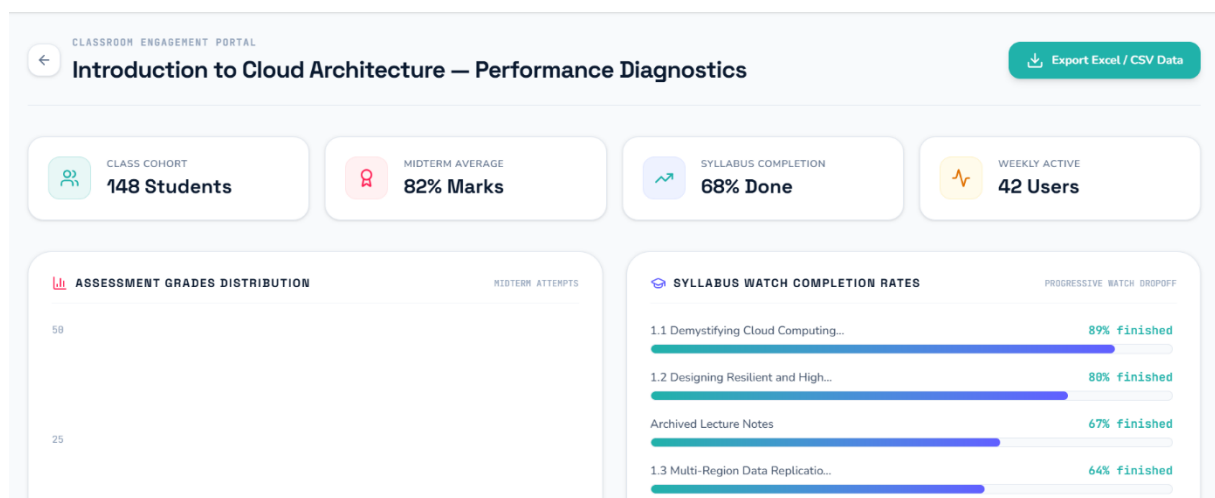


Figure 4 Course Analytics

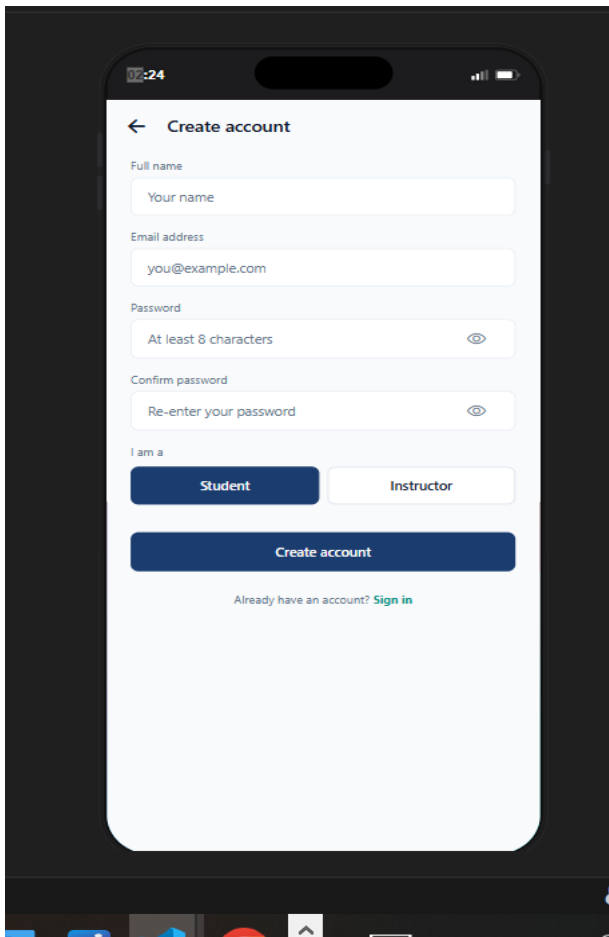


Figure 5 Signup for User

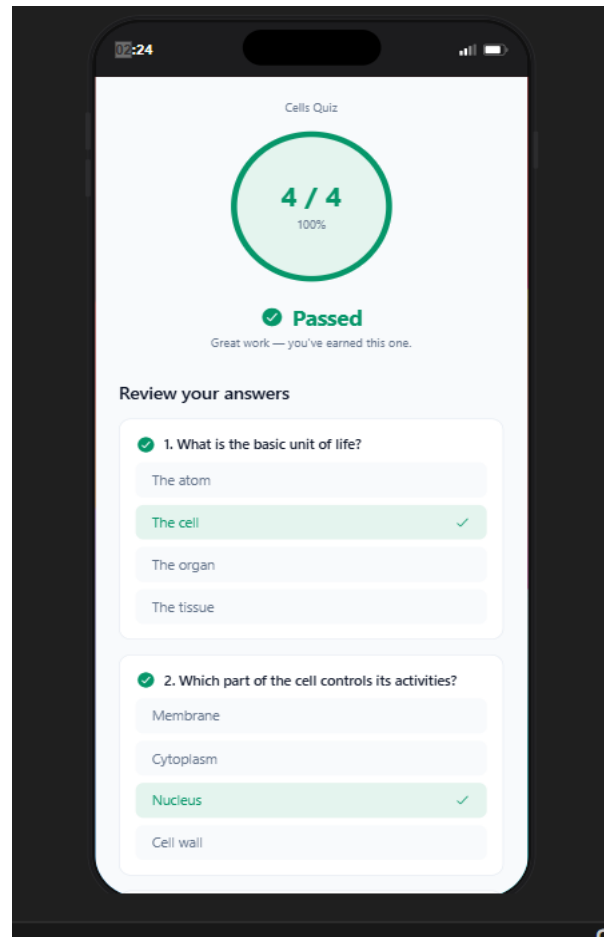


Figure 6 Quiz Section

V. CONCLUSION

The UI design and implementation of BridgeLearn provides users with an Opportunity of Improvement in their learning. The user interface closely follows the system design from Task 4 and effectively delivers the core features in a user-friendly way. This task represents a major step toward the full realization of the app's functionality and impact.

Appendix:

- Geekforgeeks for Understanding of key terms
- Figma for Figma design and prototype